

Adham Albasha

Event Booking · Weekly Mini Project 02 · Team 4

Backend

Responsibility: Bookings Module, Booking Rules and Error Handling

You own everything that changes data: creating bookings, finding them and cancelling them. This is where most business rules live, and the specification says invalid operations must be rejected even from outdated pages or repeated submissions. Every rule is enforced here, in the backend.

You provide

- The global error format and CORS setup (to everyone)
- Working booking endpoints (to Badr, Yousef, Allaith)
- Clear rejection messages for every invalid case

You depend on

- Majed: the store, the types and computeAvailability (Phase 1 and 2 of Majed)
- Allaith: the final API contract

Working rule: all business rules are enforced in the backend. If something is unclear or the contract needs a change, tell the team leader (Allaith) so every file stays consistent.

Your Flow, Phase by Phase

Phase 1: Global Setup

Set up what every endpoint, including Majed's, will share.

- ☐ Pull Majed's project foundation (store, types, time helper) and create the bookings module (controller, service, DTOs).
- ☐ Enable CORS so the React app can call the backend.
- ☐ Enable a global ValidationPipe with transform: true and whitelist: true.
- ☐ Write one global exception filter that always returns { statusCode, message }. When validation produces a list of messages, join them into one readable sentence. Unknown crashes return 500 with a generic friendly message, never a stack trace.
- ☐ Tell Majed and the frontend team that the shared error format is ready.

Phase 2: Create Booking (POST /bookings)

Implement the booking rules in a strict, fixed order.

- ☐ Body: eventId, customerName, customerPhone, places.
- ☐ Check 1, input validity (400): name is required and not just spaces; phone is valid (optional leading +, then 8 to 15 digits; ignore spaces and dashes); places is a whole number from 1 to 4. Reject 0, negatives, decimals, text and values above 4, each with a clear message.
- ☐ Check 2, event exists (404): "Event not found".
- ☐ Check 3, event not started (409): "This event has already started and can no longer be booked".
- ☐ Check 4, event not Full (409): "This event is full".
- ☐ Check 5, enough places (409): "Only N places remain for this event", using the real remaining number. Never create a partial booking.
- ☐ Check 6, duplicate (409): the same normalized phone cannot hold two Active bookings for the same event: "You already have an active booking for this event". A Cancelled booking does not block a new one. A different event is always allowed.
- ☐ Normalize the phone (remove spaces and dashes) before comparing, so "0933 123 456" and "0933123456" count as the same person.
- ☐ Create the booking: status Active, bookedAt set to now, and a unique code.
- ☐ Code generation: EV- followed by 4 random digits, retried until it does not collide with any existing code (including seeded ones); switch to 5 digits if all are used.
- ☐ Return 201 with the full Booking, including the event with its updated availability so the confirmation screen can show the new remaining places.

Phase 3: Find Booking (POST /bookings/find)

Find a booking only when both values match.

- ☐ Body: phone, code. Normalize the phone and compare the code case-insensitively.
- ☐ Return the booking only if BOTH match the same booking. If only one matches, treat it as not found (never reveal which part was wrong).
- ☐ Not found returns 404 with "Booking not found. Check your phone number and booking code".
- ☐ Every booking response includes `canCancel` computed in the backend: true only when status is Active and the event has not started. The frontend shows the Cancel button based on this value only.

Phase 4: Cancel Booking (POST /bookings/:id/cancel)

Return the places and protect against invalid cancellations.

- ☐ Body: phone. The phone must match the booking, otherwise respond as not found.
- ☐ Booking does not exist: 404 "Booking not found".
- ☐ Booking already Cancelled: 409 "This booking is already cancelled".
- ☐ Event already started: 409 "This booking can no longer be cancelled because the event has started".
- ☐ Otherwise set status to Cancelled (and store `cancelledAt`). Do not delete the booking.
- ☐ Because availability is computed from Active bookings, the places return immediately. Return the updated booking with the updated event.
- ☐ After cancelling, the same phone must be able to book the same event again if places remain. Test this explicitly.

Phase 5: Consistency and Concurrency

Make overbooking impossible, even with repeated clicks.

- ☐ Do the availability check and the creation of the booking inside one synchronous block, with no `await` between them, so two requests can never both pass the check.
- ☐ Never keep your own copy of remaining places. Always call Majed's `computeAvailability`.
- ☐ Test a repeated submission (same request sent twice quickly): the second must be rejected as a duplicate.
- ☐ Test filling an event completely, then trying again: must be rejected with "This event is full" and the event must show Full and 0 remaining.

Phase 6: Testing

Walk through every scenario from section 11 of the specification and record the result.

- ☐ Full event is booked: rejected, message says the event is full.
- ☐ More places than remain (for example 3 when 2 remain): rejected, message states how many remain.
- ☐ Places is 0, -1, 5, 2.5, or text: rejected with a clear message.
- ☐ Past event is booked: rejected. Non-existent event: "Event not found".
- ☐ Missing name, missing phone, invalid phone: rejected with a clear message each.
- ☐ Same phone books the same event twice: rejected; a different phone is accepted; the same phone on another event is accepted.
- ☐ Find with wrong phone, wrong code, or both wrong: "Booking not found".
- ☐ Cancel twice: second rejected. Cancel after the event started: rejected. Cancel then rebook: accepted.
- ☐ Share the results with Allaith and fix anything that does not match the specification exactly.

Phase 7: Presentation Preparation

Prepare your speaking part in English.

- ☐ Topic: the rules about identity, duplicate bookings and cancellation, plus the closing summary of the project.
- ☐ Prepare to demonstrate or explain one rejected case live (for example the duplicate booking) and what message the user sees.
- ☐ Practice 3 to 4 sentences per rule in your own words.

Definition of Done

- ☐ Every business rule in section 7 is enforced in the backend
- ☐ Every failure returns { statusCode, message } with a readable sentence
- ☐ No way to overbook, including repeated submissions
- ☐ Find needs both phone and code; cancel returns places immediately
- ☐ canCancel is returned in every booking response

Your Part in the Final English Presentation

Business rules: identity, duplicates and cancellation, plus the closing summary.

Appendix: API Contract Reference

Route	Input	Success	Owner
GET /events	query: category, date (YYYY-MM-DD), availableOnly=true, includePast=true	200 + Event[]	Majed
GET /events/categories	none	200 + string[]	Majed
GET /events/:id	none	200 + Event	Majed
POST /bookings	body: eventId, customerName, customerPhone, places	201 + Booking	Adham
POST /bookings/find	body: phone, code	200 + Booking	Adham
POST /bookings/:id/cancel	body: phone	200 + Booking	Adham

Event: id, title, description, date (YYYY-MM-DD), time (HH:mm), location, category, capacity, booked, remaining, bookingPercentage, status

status: "Available" | "Almost full" | "Full" | "Past"

Booking: id, code (EV-XXXX), eventId, event (full Event, with updated availability), customerName, customerPhone, places, bookedAt, status ("Active" | "Cancelled"), canCancel (boolean)

Error shape (every failure): { statusCode, message }, where message is one readable sentence ready to show to the user. Codes: 400 invalid input, 404 not found, 409 rule conflict (full, not enough places, duplicate, past event, already cancelled, event started).

This contract is a proposal. The team leader confirms the final version; if a field name changes, the leader updates this reference.